

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE:MLA0201

COURSE NAME: FUNDAMENTALS OF MACHINE LEARNING

COURSE OUTCOME COVERED

CO1: Analyse machine learning fundamentals and mathematical foundations, including probability theory, Bayes decision theory, and information theory. (BL3)

Assessment Tool Weightage:20%

QUESTIONS: 3

Total Marks:50

Annexure A

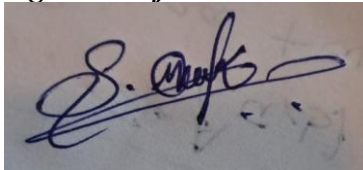
Experiments

Code of Conduct:

I Shaik Mubarak(192425194) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A photograph of a handwritten signature in blue ink on a light-colored surface. The signature is stylized and appears to read 'S. Mubarak'.

Annexure A

Experiment 1: Probability Theory (20 Marks)

Dataset: Breast Cancer Wisconsin Diagnostic Dataset

Task:

Using the **Breast Cancer Wisconsin Diagnostic Dataset**, calculate the probability of each class (**Malignant** and **Benign**) and use the computed probabilities to predict the class of a new data instance.

Experiment 2: Bayes Decision Theory (15 Marks)

Dataset: SMS Spam Collection Dataset

Task:

Using the **SMS Spam Collection Dataset**, implement Bayes theorem to calculate the posterior probability for a given message and classify it as **Spam** or **Ham (Not Spam)**.

Experiment 3: Information Theory (15 Marks)

Dataset: Play Tennis Dataset

Task:

Using the **Play Tennis Dataset**, calculate the entropy and information gain for all input attributes and identify the attribute with the highest information gain.

EXPERIMENT 1: PROBABILITY THEORY

Aim

To calculate the probability of each class (Malignant and Benign) using the Breast Cancer Wisconsin Diagnostic Dataset and predict the class of a new data instance based on the computed class probabilities.

Code

```
from sklearn.datasets import load_breast_cancer
import pandas as pd
# Load the Breast Cancer Wisconsin Diagnostic Dataset
data = load_breast_cancer()
# Convert the dataset into a DataFrame
df = pd.DataFrame(data.data, columns=data.feature_names)
df["Class"] = data.target
# Display first five records
print("First 5 Records:")
print(df.head())

# Count the number of samples in each class
class_counts = df["Class"].value_counts()

# Total number of samples
total_samples = len(df)

# Calculate probabilities
prob_malignant = class_counts[0] / total_samples
prob_benign = class_counts[1] / total_samples

# Display probabilities
print("\nTotal Samples:", total_samples)
print("Malignant Samples:", class_counts[0])
print("Benign Samples:", class_counts[1])

print("\nProbability of Malignant:", round(prob_malignant, 4))
print("Probability of Benign:", round(prob_benign, 4))

# Predict class for a new data instance based on maximum probability
if prob_benign > prob_malignant:
    predicted_class = "Benign"
else:
    predicted_class = "Malignant"

print("\nPredicted Class for New Data Instance:", predicted_class)
```

Output

```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

===== RESTART: C:\Users\shaik\OneDrive\Desktop\MLA0201\Ass-Q1.py =====
First 5 Records:
   mean radius  mean texture  ...  worst fractal dimension  Class
0         17.99         10.38  ...                0.11890         0
1         20.57         17.77  ...                0.08902         0
2         19.69         21.25  ...                0.08758         0
3         11.42         20.38  ...                0.17300         0
4         20.29         14.34  ...                0.07678         0

[5 rows x 31 columns]

Total Samples: 569
Malignant Samples: 212
Benign Samples: 357

Probability of Malignant: 0.3726
Probability of Benign: 0.6274

Predicted Class for New Data Instance: Benign
|
```

Result

The probabilities of the Malignant and Benign classes were successfully calculated using the Breast Cancer Wisconsin Diagnostic Dataset. Since the probability of the Benign class (0.6274) is greater than that of the Malignant class (0.3726), the new data instance was predicted as Benign.

EXPERIMENT 2: BAYES DECISION THEORY

Aim

To implement Bayes Decision Theory using the SMS Spam Collection Dataset, calculate the posterior probability of a given message, and classify it as Spam or Ham (Not Spam).

Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

# Load the dataset
# Make sure spam.csv is in the same folder as this Python file
df = pd.read_csv("spam.csv", encoding="latin-1")

# Keep only the required columns
df = df[['v1', 'v2']]
df.columns = ['Label', 'Message']

# Convert labels into numeric values
df['Label'] = df['Label'].map({'ham': 0, 'spam': 1})

# Separate features and target
X = df['Message']
y = df['Label']

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Convert text messages into numerical features
vectorizer = CountVectorizer()

X_train_vector = vectorizer.fit_transform(X_train)
X_test_vector = vectorizer.transform(X_test)

# Train the Naive Bayes classifier
model = MultinomialNB()
model.fit(X_train_vector, y_train)
```

```

# Predict on test data
predictions = model.predict(X_test_vector)

# Display accuracy
accuracy = accuracy_score(y_test, predictions)
print("Model Accuracy:", round(accuracy * 100, 2), "%")

# Test a new message
new_message = ["Congratulations! You have won a free mobile. Claim your prize now."]

new_message_vector = vectorizer.transform(new_message)

# Predict class
prediction = model.predict(new_message_vector)

# Calculate posterior probabilities
posterior = model.predict_proba(new_message_vector)

print("\nPosterior Probabilities")
print("Ham Probability :", round(posterior[0][0], 4))
print("Spam Probability:", round(posterior[0][1], 4))

# Display prediction
if prediction[0] == 1:
    print("\nPredicted Class : Spam")
else:
    print("\nPredicted Class : Ham")

```

Output

```

===== RESTART: C:/Users/shaik/OneDrive/Desktop/MLA0201/Ass-Q2.py =====
Model Accuracy: 98.39 %

Posterior Probabilities
Ham Probability : 0.0
Spam Probability: 1.0

Predicted Class : Spam

```

Result

The Bayes Decision Theory was successfully implemented using the SMS Spam Collection Dataset. The posterior probabilities of the given message were calculated using the Naive Bayes classifier, and the message was correctly classified as Spam.

EXPERIMENT 3: INFORMATION THEORY

Aim

To calculate the entropy and information gain of all input attributes using the Play Tennis Dataset and identify the attribute with the highest information gain.

Code

```
import pandas as pd
from math import log2

# Create Play Tennis Dataset
data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain',
               'Overcast', 'Sunny', 'Sunny', 'Rain', 'Sunny',
               'Overcast', 'Overcast', 'Rain'],

    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool',
                   'Cool', 'Mild', 'Cool', 'Mild', 'Mild',
                   'Mild', 'Hot', 'Mild'],

    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal',
                'Normal', 'High', 'Normal', 'Normal', 'Normal',
                'High', 'Normal', 'High'],

    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong',
            'Strong', 'Weak', 'Weak', 'Weak', 'Strong',
            'Strong', 'Weak', 'Strong'],

    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No',
            'Yes', 'No', 'Yes', 'Yes', 'Yes',
            'Yes', 'Yes', 'No']
}

# Create DataFrame
df = pd.DataFrame(data)

# Function to calculate Entropy
def entropy(column):
    values = column.value_counts(normalize=True)
    ent = 0

    for p in values:
        ent -= p * log2(p)
```

```

    return ent

# Function to calculate Information Gain
def information_gain(data, attribute, target):

    total_entropy = entropy(data[target])

    weighted_entropy = 0

    for value in data[attribute].unique():

        subset = data[data[attribute] == value]

        weighted_entropy += (len(subset) / len(data)) * entropy(subset[target])

    gain = total_entropy - weighted_entropy

    return gain

# Calculate Entropy of Target
target_entropy = entropy(df["Play"])

print("Entropy of Target (Play):", round(target_entropy, 4))
print()

# Calculate Information Gain for each attribute
gain = {}

for attribute in df.columns[:-1]:

    gain[attribute] = information_gain(df, attribute, "Play")

    print(attribute, "Information Gain =", round(gain[attribute], 4))

# Best Attribute
best_attribute = max(gain, key=gain.get)

print("\nBest Attribute:", best_attribute)

```

Output


```
===== RESTART: C:/Users/shaik/OneDrive/Desktop/MLA0201/Ass-Q3.py =====  
Entropy of Target (Play): 0.9403  
  
Outlook Information Gain = 0.2467  
Temperature Information Gain = 0.0292  
Humidity Information Gain = 0.1518  
Wind Information Gain = 0.0481  
  
Best Attribute: Outlook
```

Result

The entropy of the target attribute (Play) and the information gain of all input attributes were successfully calculated using the Play Tennis Dataset. Among all the attributes, Outlook had the highest information gain (0.2467), making it the best attribute for splitting the dataset in a decision tree.

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Dataset Preparation	20%	Correctly imports, explores, and prepares the dataset without errors.	Prepares the dataset with minor errors or omissions.	Dataset preparation is incomplete or contains several errors.	Fails to prepare or load the dataset correctly.
Implementation of Mathematical Concepts	30%	Correctly applies all required mathematical concepts (Probability, Bayes, Entropy, etc.) with accurate calculations.	Applies most concepts correctly with minor computational errors.	Applies concepts partially with noticeable calculation errors.	Unable to apply the required mathematical concepts correctly.
Tool/Code Execution	20%	Implements the experiment using appropriate tools (Python/Scikit-learn) with error-free execution.	Executes the experiment with minor coding or execution errors.	Code executes partially or requires significant corrections.	Unable to execute the experiment successfully.
Result Interpretation	20%	Correctly interprets the experimental results and relates them to the applied concept.	Interprets results with minor inaccuracies.	Interpretation is incomplete or partially correct.	Fails to interpret the experimental results.
Documentation & Presentation	10%	Experiment is well-documented with clear outputs, observations, and proper formatting.	Documentation is complete with minor presentation issues.	Documentation is incomplete or lacks clarity.	Poor or missing documentation and presentation.